Bank Respublika
BA Interview Guide

Their Pain Points + Your Solutions from Embafinans

What to Say, When to Say It, How to Win

Zamir Jamalov

IT Business Analyst | Fintech and E-Commerce

Table of Contents

1. The Game Plan

This guide is your weapon for the Bank Respublika interview. It is not about listing what you did at Embafinans. It is about showing them that you have ALREADY solved the exact problems they are facing RIGHT NOW. Every bank has the same problems. You have already fixed these problems. Your job in the interview is to make them see this.

The strategy has three parts. First, understand what pain points Bank Respublika probably has. Second, map each pain point to what you did at Embafinans. Third, know exactly WHEN in the conversation to bring up each pain point. Timing is everything.

1.1. The Core Message

Your core message is simple: "I am not just a BA who writes documents. I am a problem solver. I have been inside the chaos of business teams fighting with IT teams, and I know how to fix it. At Embafinans, I fixed these problems. I can fix them here too." This is what you want them to think after every answer you give.

> **Remember:** Do not say "I wrote a BRD." Nobody cares. Say "The business team and IT team were fighting because nobody knew what to build. I wrote a BRD with REQ numbers so everybody could see the same requirements. The fighting stopped." THAT is what they care about.

2. Bank Respublika Pain Points

Before the interview, you need to understand what problems Bank Respublika probably faces. You do not know their exact situation, but EVERY bank has these problems. If you walk in and show that you already solved these problems at Embafinans, you will win.

2.1. Pain Point #1: Business and IT Do Not Understand Each Other

This is the #1 problem in every bank. The business team says "we need this feature." The IT team says "that is not possible." They argue. Nothing happens for weeks. Projects get delayed. At Embafinans, this happened EVERY DAY. The risk team wanted one thing, the developers said another thing. I was the person in the middle who fixed this.

> **Your story:** "At Embafinans, the risk team said we need to check every loan application manually. The developers said that is too slow, the system cannot handle it. They were both right, but they could not talk to each other. I sat with risk to understand their rules. Then I sat with developers to understand their limits. Then I designed a pre-screen model that filtered out 90% of applications automatically. Risk was happy because they only reviewed real candidates. Developers were happy because the load was 10x less. This is what I do - I translate between business and IT."

2.2. Pain Point #2: Requirements Are Not Clear

When requirements are not clear, developers build the wrong thing. Then you have to redo it. This wastes time and money. At Embafinans, I saw this problem on my first day. Developers were asking "what exactly do you want?" and nobody could answer. I fixed this by creating structured documents that left no room for confusion.

> **Your story:** "When I joined Embafinans, there was no proper documentation. A business person would say something in a meeting, and three different developers would understand it three different ways. I introduced REQ-101 numbering. Every requirement got a unique number - REQ-101, REQ-102, REQ-103. I wrote User Stories with Gherkin Acceptance Criteria. After that, when Developer A said he finished REQ-105, everybody could check the same document and confirm. No more confusion. No more building the wrong thing."

2.3. Pain Point #3: Projects Get Delayed

Banks have deadlines. Regulatory deadlines, business deadlines, competitor pressure. But projects keep getting delayed. Why? Because there is no clear plan. Scope keeps growing. Nobody is watching the timeline. At Embafinans, I delivered 4 production projects on time. Here is how.

> **Your story:** "I delivered 4 projects at Embafinans - BNPL Credit Scoring, B2C Sales Channel, Delivery Tracking Dashboard, and Credit Lifecycle. All 4 went live on time. How? Two things. First, I used RICE framework for backlog prioritization. Every feature was scored on Reach, Impact, Confidence, and Effort. So when someone said add this new thing, I could show the numbers: this new thing has low impact but high effort, so we do it next sprint, not this sprint. Second, I ran structured UAT with clear test scenarios. When stakeholders tested, they knew exactly what to test. No surprises. On-time sign-off."

2.4. Pain Point #4: Stakeholders Have Conflicting Priorities

In a bank, you have risk, compliance, sales, operations, IT, finance - all these teams want different things. Risk wants strict rules. Sales wants easy rules. Operations wants automation. IT wants clear specs. These priorities conflict. Most BAs just listen and write down everything. I do something different - I use data to resolve conflicts.

> **Your story:** "At Embafinans, risk wanted to make credit scoring stricter. Sales wanted to keep it easy. They argued for two weeks. Nothing moved. I took both sides data into SQL. I ran analysis and found that making scoring 10% stricter would increase rejection rate by 15% but reduce default risk by only 2%. The numbers showed it was not worth it. Risk team looked at the data and agreed. Sales team was happy. I did not use opinions. I used SQL data. When people see numbers, they stop arguing."

2.5. Pain Point #5: UAT Takes Forever

User Acceptance Testing is supposed to be the final check before go-live. But in many banks, UAT becomes a nightmare. Business testers do not know what to test. They find bugs at the last minute. Everything gets delayed. I fixed this at Embafinans by making UAT structured and predictable.

> **Your story:** "At Embafinans, UAT used to be chaotic. Business people would come in, click around randomly, find a bug, and we would have to go back to development. I changed this. Before UAT, I prepared test scenarios in Gherkin format - Given, When, Then. Business testers knew exactly what to test and what result to expect. I also ran bug triage meetings after each UAT round - QA, developers, and business sat together.

We categorized every bug as Critical, Major, or Minor. Critical bugs were fixed immediately. Minor bugs went to the next release. This way, UAT finished on schedule every time."

2.6. Pain Point #6: No Clear Process

Many banks do not have clear processes. When a new project starts, nobody knows the steps. Who approves what? Who tests what? What is the workflow? Without a clear process, people waste time figuring out basic things. I solved this at Embafinans by mapping every process.

Your story: "When I started the Goods Loan project, the delivery process was a mess. No one knew the exact steps from loan approval to item delivery. I sat with the operations team for three days and watched how they work. Then I drew a BPMN diagram - first the As-Is process, which had 12 steps. Then I designed the To-Be process, which had 7 steps. Operations could see exactly what would change. They approved it immediately because they could see the full picture. BPMN is powerful because everyone - business, IT, management - can understand it."

2.7. Pain Point #7: Vendor Integration Problems

Banks work with many external vendors - payment providers, e-signature services, scoring bureaus, etc. Integrating with these vendors is always problematic. API documentation is unclear, data formats do not match, testing is hard. At Embafinans, I handled two big vendor integrations: payment gateway and e-signature provider.

Your story: "For the B2C Sales Channel, we needed to integrate with a payment gateway. The vendor gave us API documentation, but it was 200 pages and nobody had time to read it. I read the whole thing. I created a data mapping document that showed exactly which field from our system maps to which field in the vendor system. I wrote API specs in Swagger so our developers could see the endpoints, request formats, and response formats in a clean way. The integration was completed in 2 weeks instead of the expected 6 weeks, because there were no questions left unanswered. For the e-signature integration, I did the same thing - studied the vendor API, created specs, and our developers built it without any confusion."

2.8. Pain Point #8: The BA Does Not Understand IT

This is a secret pain point. Many BAs in banks come from pure business backgrounds. They can write good business requirements, but when developers say "the database index is slow" or "this API call is synchronous and blocks the thread," the BA has no idea what they mean. This creates a gap. The developer has to explain technical things in simple words, which wastes time. Sometimes requirements get changed because the BA did not understand the technical limitation. This is YOUR biggest advantage - you have 15 years of technical background.

Your story: "I have 15 years of software engineering experience - C# backend, databases like Oracle, MSSQL, PostgreSQL, MongoDB, system integration, CI/CD pipelines. I worked at Central Bank of Azerbaijan, Unibank, and ASAN Service. When a developer tells me there is a performance issue, I understand WHY. When they say the API response time is too slow because of the database query, I can suggest indexing or caching because I have done it myself. I am not a BA who only writes business requirements. I am a BA who understands the code. This means there is no gap between me and the development team. We speak the

same language."

3. Quick Reference: Pain Point to Solution Map

This table is your cheat sheet. Study it before the interview. When they describe a problem, your brain should immediately connect it to your Embafinans experience.

Their Pain Point	What You Did at Embafinans	Key Words to Use
Business and IT cannot communicate	Risk team vs developers on BNPL scoring - I translated between both sides, designed pre-screen model	Bridge, translate, pre-screen model, both sides happy
Requirements are not clear	REQ-101 numbering, BRD with Gherkin Acceptance Criteria, User Stories, no more confusion	REQ-101, Gherkin, traceability, zero confusion
Projects get delayed	4 projects delivered on time, RICE prioritization, structured UAT, on-time sign-off	On-time, RICE, sprint, sign-off, 4 production projects
Conflicting stakeholder priorities	SQL data analysis resolved risk vs sales conflict, evidence-based decisions	SQL, data-driven, evidence-based, consensus
UAT is chaotic	Gherkin test scenarios, bug triage meetings, Critical/Major/Minor classification	Gherkin, bug triage, structured testing, predictable
No clear process	BPMN As-Is/To-Be diagrams, 12 steps reduced to 7, operations approved immediately	BPMN, As-Is, To-Be, process optimization
Vendor integration problems	Payment gateway + e-signature integration, data mapping, Swagger API specs, 2 weeks instead of 6	Swagger, data mapping, vendor API, integration spec
BA does not understand IT	15 years dev experience, C# / Oracle / PostgreSQL / MongoDB / CI/CD, speak same language as devs	15 years, technical BA, no gap, same language

4. When to Say What - Timing Strategy

This is one of the most important parts of this guide. It is not enough to know WHAT to say. You need to know WHEN to say it. If you talk about the right things at the wrong time, it sounds random. But if you bring up a pain point RIGHT AFTER they mention a similar problem, they will think you are reading their mind.

4.1. The Interview Conversation Flow

A typical BA interview has several phases. Here is how you should handle each phase:

Interview Phase	What They Will Ask	What You Should Do	Pain Points to Mention
1. Opening (first 2 min)	Tell me about yourself / walk me through your CV	Do NOT read your CV. Tell a story. Start with your 15 years technical background, then explain WHY you became a BA. End with your core value: I bridge business and IT.	Pain #8 (BA does not understand IT), Pain #1 (communication gap)

2. Experience deep dive (10-15 min)	Tell me about your projects / what was your biggest challenge	Pick your BEST project. Tell the full STAR story. Focus on the PROBLEMS you solved, not the features you built.	Pain #1, #2, #3, #4 - pick the ones that match the project
3. Technical questions (10 min)	How do you write BRD? What tools do you use? How do you do UAT?	Show your methodology. Mention REQ-101, Gherkin, Swagger, BPMN, SQL, RICE. Show that you are systematic.	Pain #2 (clear requirements), #5 (UAT structure), #6 (clear process)
4. Behavioral questions (10 min)	How do you handle conflict? Tell me about a time you failed	THIS IS YOUR MOMENT. Talk about stakeholder conflicts you resolved. Show that you use DATA not emotions to solve problems.	Pain #4 (conflicting priorities), #1 (communication gap)
5. Their turn to talk (5-10 min)	We have this project / We are facing this challenge...	LISTEN carefully. When they describe a problem, connect it to your Embafinans experience. Say: I had the same problem at Embafinans, and here is how I solved it.	ALL pain points - use whichever matches their problem
6. Closing (2-3 min)	Do you have questions for us?	Ask SMART questions that show you understand their problems. Do NOT ask about salary or benefits yet.	Reinforce your core message one last time

4.2. The Golden Rule of Timing

The golden rule is: listen first, then connect. When they describe a problem they are having, do NOT immediately jump to your story. First, show that you understand their problem. Say something like: "Yes, I have seen that problem many times." Then pause for one second. Then say: "At Embafinans, we had the exact same issue." This two-step approach makes them feel that you truly understand their world, not just that you are selling yourself.

Example: They say "Our requirements keep changing and projects get delayed." Do NOT say: "I used RICE framework." DO say: "Yes, that is very common. Scope creep kills timelines." (pause) "At Embafinans, I solved this with RICE framework..." Now they are listening because you showed empathy first.

4.3. When They Ask "Why Do You Want to Work Here?"

Your answer: "Because I have already solved the problems you are facing. At Embafinans, I dealt with the same challenges - business and IT not understanding each other, unclear requirements, project delays, stakeholder conflicts. I fixed all of these. I want to bring the same approach here. I am not starting from zero - I already know what works."

Notice: you did NOT say "because Bank Respublika is a great company." You showed that you understand their problems and you have solutions. This is 100x more powerful.

5. Real Problems You Faced at Embafinans

This section is about the REAL problems you faced when working with business teams and IT teams at Embafinans. These are not textbook examples. These actually happened. Interviewers can tell the difference between a real story and a fake one. The more specific details you give, the more believable your story is.

5.1. Problem: Risk Team Changed Requirements Every Week

The situation: During the BNPL Credit Scoring project, the risk team kept changing the scoring rules. Every Monday, they would come with new rules. The developers were frustrated because they had already built last week version. The project was going nowhere.

What you did: You scheduled a formal requirement freeze meeting. You explained to risk that every change costs development time. You proposed a solution: gather ALL rules in one workshop, prioritize them with RICE, and lock them for the sprint. Risk agreed. After the workshop, you documented everything with REQ numbers. No more changes during the sprint.

How to tell it: "Risk team was changing requirements every week. Developers were going crazy. I organized a workshop with risk, we listed ALL their rules, prioritized with RICE, and agreed on a sprint freeze. After that, no changes during sprint. Developers could focus on building instead of rebuilding. Risk was happy because all their rules were in the plan, just in the right order."

5.2. Problem: Developers Said the API Spec Was Not Enough

The situation: For the B2C Sales Channel, you gave developers an API spec. But they came back with 30 questions. The spec was too high-level. It did not show data types, error codes, or field formats. Developers could not start coding because they were not sure what to build.

What you did: You created a detailed Swagger/OpenAPI 3.0 spec with every endpoint, every field, every data type, every possible error response. You also created a data mapping document showing which internal field maps to which vendor field. You drew sequence diagrams showing the full flow: user action, frontend request, backend processing, vendor API call, response, and display. After this, developers had zero questions.

How to tell it: "First version of my API spec was too simple. Developers came back with 30 questions. That was my wake-up call. I rewrote everything in Swagger/OpenAPI 3.0 with full detail - endpoints, data types, error codes, examples. Plus a data mapping document and sequence diagrams. After that, developers started coding immediately. Zero questions. That taught me: as a BA, the quality of your spec directly affects developer speed."

5.3. Problem: Operations Team Refused the New System

The situation: For the Delivery Tracking Dashboard, the operations team did not want to use the new system. They were used to Excel. They said the new system was too complicated. The project was at risk because if operations does not use it, the whole investment is wasted.

What you did: Instead of forcing the new system, you sat with operations for three days and watched how they work. You drew BPMN diagrams of their current process. You showed them the As-Is diagram - they saw how

messy it was. Then you showed the To-Be diagram - 12 steps reduced to 7. You asked for their feedback on each step. They felt included in the design. When the dashboard launched, they adopted it immediately because they helped design it.

How to tell it: "Operations refused to use the new dashboard. I did not argue. I sat with them for three days and watched their work. I drew BPMN diagrams - their current process had 12 steps. I showed them a new version with 7 steps. But I did not just show it - I asked for their input on every step. They felt ownership. When we launched, they were the biggest supporters. Lesson: never design a system FOR users. Design it WITH users."

5.4. Problem: Sales and Risk Could Not Agree on Credit Rules

The situation: For the End-to-End Credit Lifecycle project, sales wanted to approve more loans (more revenue). Risk wanted to approve fewer loans (less risk). They argued for two weeks. The project was stuck. Nobody could make a decision because both sides had valid points.

What you did: You took both sides data into SQL. You analyzed historical loan data. You found that 90% of rejected applications were from a specific risk category that actually had very low default rate. You showed the data to risk: these applications are safe, rejecting them does not reduce risk, it only reduces revenue. Risk agreed to change the rule. Sales got more approvals. Both sides were happy because the decision was based on data, not opinions.

How to tell it: "Sales wanted more approvals, risk wanted fewer. Two weeks of arguing. I took all historical loan data into SQL and found something interesting: 90% of rejected applications were actually low-risk. I showed the numbers to risk team. They looked at the data and agreed to change the rule. Both sides got what they wanted. But the key point is: I did not pick a side. I used data to find the truth. That is what a BA should do - be neutral, be data-driven."

5.5. Problem: UAT Found Critical Bugs One Day Before Launch

The situation: During BNPL project UAT, stakeholders found 3 critical bugs on the last day of testing. The launch was scheduled for the next day. Everyone panicked. Management asked: can we still launch? Developers said: we need 3 more days. Business said: we cannot delay, marketing is ready.

What you did: You immediately called a bug triage meeting. You categorized the 3 bugs: Bug 1 was critical but only affected 2% of users. Bug 2 was actually a duplicate of Bug 1. Bug 3 was not a bug at all - it was expected behavior. So there was really only 1 real critical bug. You proposed a compromise: launch for 98% of users, fix Bug 1 in parallel, and deploy the fix within 24 hours. Everyone agreed. Launch happened on time.

How to tell it: "One day before go-live, stakeholders found 3 critical bugs. Panic. I called a bug triage meeting immediately. I analyzed each bug: one was real but only affected 2% of users, one was a duplicate, one was not actually a bug. So really there was only 1 bug. I proposed: launch for 98% of users, fix the remaining bug in parallel. Everyone agreed. Lesson: in a crisis, do not panic. Analyze, categorize, and propose a solution. That is the BA role in production situations."

6. Ready-Made Phrases for Each Interview Phase

Below are ready-made phrases you can use in each phase of the interview. These are written in simple English. Memorize the key ideas, do not memorize word for word. You need to sound natural, not like you are reading a script.

6.1. Phase 1: "Tell Me About Yourself"

"I have 15 years of experience in software engineering. I worked at Central Bank of Azerbaijan, Unibank, and ASAN Service. I built backend systems, databases, and integrations. Three years ago, I moved to Business Analysis because I saw that the biggest problem in IT projects is not technology - it is communication. Business people and IT people speak different languages. I can speak both because I lived in both worlds. At Embafinans, I delivered 4 production projects - credit scoring, online sales channel, delivery tracking dashboard, and end-to-end credit lifecycle. In every project, my main value was connecting business needs with technical solutions."

6.2. Phase 2: "What Was Your Biggest Project?"

"My biggest project was the end-to-end credit lifecycle at Embafinans. This was cross-functional - risk, sales, operations, and IT all involved. The challenge was that each team had different priorities and they conflicted. Risk wanted strict rules, sales wanted easy process. I resolved every conflict using SQL data analysis. I also introduced RICE framework for prioritization - every feature was scored on Reach, Impact, Confidence, and Effort. No more arguments about what to build first. The numbers decided. This project went live on time because there was no confusion, no wasted time, no emotional decisions. Everything was data-driven."

6.3. Phase 3: "How Do You Handle Conflicting Requirements?"

"First, I listen to both sides carefully. I do not take sides. Then I do something most BAs cannot do - I go to the data. I use SQL to analyze the situation. For example, at Embafinans, risk and sales had conflicting requirements. I pulled the historical data and found that 90% of rejected applications were actually low risk. When I showed the numbers, both sides agreed immediately. Data removes emotions from the conversation. That is my approach - be neutral, be data-driven, find the truth."

6.4. Phase 4: "Tell Me About a Difficult Situation"

"The most difficult situation was when operations team refused to use the new dashboard. They were comfortable with Excel and did not want change. I did not force them. Instead, I spent three days with them, watching how they work. I drew BPMN diagrams showing their current 12-step process and proposed a new 7-step process. But the key was: I asked for their feedback at every step. They felt like they designed the new process, not me. When we launched, they were the biggest supporters. This taught me that change management is not about pushing people - it is about including them."

6.5. Phase 5: When They Describe Their Problem

This is the most important phase. When they start telling you about THEIR problems, listen carefully. Then use one of these connectors:

> "Yes, I understand that problem. At Embafinans, we faced the exact same challenge..."
>
> "That is very common in banking. Let me tell you how I solved it at Embafinans..."
>
> "I have seen this before. The root cause is usually [X]. At Embafinans, I fixed it by..."
>
> "This is actually one of my strengths. Let me give you a specific example from Embafinans..."

Important: After they describe their problem, do NOT say "I know how to fix that." That sounds arrogant. Say "I have dealt with that before" - it sounds experienced and humble.

6.6. Phase 6: "Do You Have Questions for Us?"

Do NOT ask about salary, benefits, or working hours. Ask questions that show you understand their challenges:

> "What is the biggest challenge your IT business team is facing right now?"
>
> "How does your business team currently communicate requirements to the development team?"
>
> "What was the reason your last BA project got delayed, if it did?"
>
> "How do you handle scope creep in your current projects?"

Notice: every question you ask shows that you understand their problems. When you ask "how do you handle scope creep?" they think: this person has dealt with scope creep before. That is exactly what you want.

7. Why You Are Different from Other Candidates

Most BA candidates will walk into the interview and say the same things: "I can write BRD, I can create user stories, I know Jira, I did agile." These are basic skills. Every BA has them. Here is what makes you different - and you need to make sure they understand this.

Most BAs Say	You Say	Why It Matters
"I write BRDs."	"I write BRDs with REQ-101 numbering, Gherkin acceptance criteria, and full traceability. At Embafinans, this eliminated all confusion between business and developers."	Shows specificity and real impact
"I do stakeholder management."	"I resolved a 2-week conflict between risk and sales using SQL data analysis. When people see numbers, they stop arguing."	Shows problem-solving, not just talking
"I know agile/scrum."	"I use RICE framework for backlog prioritization. This means no more emotional debates about what to build first. Numbers decide."	Shows systematic approach
"I coordinate UAT."	"I prepare Gherkin test scenarios before UAT. I run bug triage meetings. I classify bugs as Critical/Major/Minor. On-time sign-off every time."	Shows process and accountability

"I understand business needs."	"I have 15 years of software engineering experience. I speak the same language as developers. There is zero gap between me and IT."	Shows unique technical advantage
"I do process modeling."	"I draw BPMN As-Is/To-Be diagrams. At Embafinans, this reduced a 12-step process to 7 steps and operations approved immediately."	Shows measurable improvement

8. Final Checklist - Before the Interview

Read this page the night before the interview. Make sure you are ready.

8.1. Know These Numbers by Heart

4 production projects delivered at Embafinans

BNPL: 2x faster credit decisions

B2C Sales: 300-500 daily online applications

Dashboard: 2x fewer errors, 12 steps to 7 steps

Credit Lifecycle: end-to-end (application to collection)

15 years of software engineering experience

8.2. Know These Keywords

REQ-101, BRD, FRD, SRS, User Stories, Gherkin, Given-When-Then

Swagger / OpenAPI 3.0, REST API, Data Mapping, Sequence Diagram

BPMN, As-Is, To-Be, Process Optimization

RICE Framework - Reach, Impact, Confidence, Effort

SQL Data Analysis, Evidence-Based Decisions

UAT, Bug Triage, Critical / Major / Minor, On-Time Sign-Off

Cross-Functional Teams, Stakeholder Management

Technical BA, Bridge, No Gap, 15 Years Dev Background

8.3. Remember the Timing

Phase 1 (Opening): Show you are a technical BA with 15 years background

Phase 2 (Experience): Tell your BEST project story with problems and solutions

Phase 3 (Technical): Show methodology - REQ-101, Gherkin, Swagger, BPMN

Phase 4 (Behavioral): This is YOUR MOMENT - talk about conflicts you resolved

Phase 5 (Their problems): LISTEN first, then connect to Embafinans

Phase 6 (Closing): Ask smart questions about THEIR challenges

8.4. The Last Thing They Should Remember

Your final impression: "I am not just a BA who documents requirements. I am a problem solver who happens to use BA tools. At Embafinans, I solved real problems: team conflicts, unclear requirements, project delays, process chaos. I have 15 years of technical background so I can speak to developers in their language. And I can speak to business in their language. I am the bridge. And I am ready to build that bridge here at Bank Respublika."